Sentrix

A Layer-One Blockchain for the Real Economy

Satya Kwok
satya@sentrixchain.com
sentrixchain.com

Version 1.2.4 *(final unless chain hard-fork)*

Abstract

We propose Sentrix, a Layer-One blockchain optimized for real-world economic settlement. Sentrix uses a delegated-proof-of-stake validator selection paired with a three-phase Byzantine Fault Tolerant agreement protocol to produce one-second-final blocks. Native protocol operations—token issuance, staking, validator coordination—execute directly against canonical state, while a second execution rail runs the Ethereum Virtual Machine for general-purpose programmability. Monetary policy is fixed: a maximum supply of 315 million SRX, a one-SRX block reward halving every approximately 126 million blocks (four years at one-second blocks), and a fee mechanism that destroys 50% of every transaction fee forever. The chain is designed for durable financial infrastructure for real-world economic activity, beginning in Indonesia and scaling outward. This paper specifies the design rationale, architecture, transaction lifecycle, consensus safety properties, monetary mechanics, and threat model.

Focus Statement

Sentrix is financial infrastructure for the real economy. Every design choice in this paper—sub-second finality, native payment primitives, EVM compatibility, capped halving deflationary supply, open-permissionless validator set—serves one goal: making payment, savings, asset transfer, and contract settlement work for actual businesses and households, beginning with Indonesia.

Sentrix is not a speculation venue. It is not a DeFi-first chain. It is not a rollup framework. It is not a high-frequency-trading substrate. It is a settlement rail for the kind of economic activity that already exists in the physical world—remittances, retail payments, supplier invoicing, cooperative savings, real-world-asset tokenization, cross-border commerce—and that has been poorly served by both the legacy banking system and the speculation-oriented public chains that came before us.

The chain's economic constants (315M cap, four-year halving, 50% fee burn, genesis allocation) are non-governable: no proposal, no fork, no upgrade can change them. They are the foundational economic contract between the protocol and its users, and they are stable for as long as the chain operates.

1 Introduction

Most public blockchains were designed to do something other than serve real economic activity. Bitcoin was designed to be a peer-to-peer digital cash system [1], but in practice operates

primarily as a settlement layer for value storage. Ethereum was designed as a world computer [2], but its on-chain economy is dominated by financial speculation, on-chain trading, and synthetic assets. Layer-Two scaling solutions inherit these orientations and amplify them.

The disconnect between blockchain infrastructure and the actual economic activity of the majority of the world's population is striking. The bulk of human commerce remains denominated in local currencies, settled through banking rails that have not materially improved in decades, and effectively excluded from public-blockchain participation. The friction is not philosophical—real businesses, real cooperatives, real cross-border merchants want fast and final settlement—but architectural. The infrastructure they would need is not the infrastructure that has been built.

Sentrix is a response to this architectural gap. It is a blockchain whose every design choice prioritizes real economic settlement over trading speculation. Where general-purpose chains favor maximum flexibility at the cost of efficiency, Sentrix promotes the most common economic primitives to native protocol operations, eliminating an entire class of overhead. Where most chains tolerate ten- or thirty-second block times, Sentrix targets one-second finality, sufficient for retail-grade interactions. Where many chains follow inflationary token models, Sentrix is hard-capped, halving, and deflationary—every fee is partially destroyed forever, so circulating supply contracts as activity grows.

Sentrix is built first for Indonesia, where the population, unbanked share, and remittance volume present a uniquely large unmet demand. From there it is intended to scale outward.

2 Vision, Mission, and Rationale

2.1 Vision

A future in which real-world economic activity—cross-border payments, asset-backed lending, retail commerce, cooperative savings, supply-chain settlement, identity verification—runs on rails that are transparent, low-cost, programmable, not owned by a single corporation, accessible from any internet connection, and as fast as a credit-card swipe.

We do not believe this vision can be realized through retrofits of legacy banking systems. We do not believe it can be realized on blockchains designed for trading speculation. It can be realized through purpose-built infrastructure that treats real-economy settlement as a first-class concern rather than an afterthought.

2.2 Mission

Build the most reliable, lowest-cost, most accessible settlement layer for real-world economic activity, starting from the geographies and use cases that have been failed by legacy infrastructure.

Concretely, this means:

- One-second final settlement for any transfer, swap, or contract call.
- Transaction fees measured in fractions of a cent, regardless of transfer size.
- Native protocol support for token issuance, staking, and asset management without requiring smart-contract deployment.
- Compatibility with the global Ethereum developer toolchain so existing applications can port without modification.

- Open codebase, transparent operations, deterministic monetary policy.
- Market-entry orientation toward Indonesia and Southeast Asia, where unmet demand is largest.

2.3 Why Sentrix Exists

The infrastructure problem we address has four dimensions:

Latency. Existing payment rails settle in days (correspondent banking), hours (card networks), or tens of seconds (blockchains optimized for security or trading). Real commerce demands sub-second confirmations for point-of-sale, retail, and routine business operations.

Cost. Cross-border remittance fees of 5–10% remain common. Card-network interchange fees of 1–3% extract value at every retail transaction. Smart-contract gas costs on general-purpose chains routinely exceed the value being transferred for small payments. Sentrix's native operations target costs measured in fractions of a US cent regardless of the underlying asset's value.

Programmability without overhead. Blockchains have demonstrated that programmable transactions enable powerful new economic constructions—lending, escrow, automated market making, identity verification. But routine operations like transferring tokens, claiming staking rewards, or registering validators should not require deploying audited smart-contract code. They should be part of the protocol.

Inclusion. Banking infrastructure correlates with national wealth. Public-blockchain infrastructure could in principle break this correlation, but in practice has reproduced it: most on-chain activity originates from a small set of high-income jurisdictions. Sentrix is positioned to invert this default by building first for the geographies most underserved by legacy systems.

2.4 Why Now

Three preconditions have converged that make this work feasible.

First, the EVM has matured into a standard. Tooling, wallets, smart-contract libraries, security-audit conventions, and developer mental models have converged. New chains adopting EVM compatibility inherit the entire ecosystem at zero cost.

Second, Byzantine Fault Tolerant consensus has been productionized. Tendermint-style BFT has run reliable mainnets for years. The hard part of consensus is now well-trodden, and new chains can build on robust open-source implementations.

Third, Rust has matured into a viable systems language for blockchain implementation. Memory-safety guarantees, mature async runtime, performant cryptographic libraries, and a culture of zero-copy efficiency make solo or small-team chain development feasible in ways it was not a decade ago.

A chain combining all three—EVM compatibility, BFT consensus, and a Rust implementation—and applying them to real-economy use cases is technically feasible today in a way it was not five years ago.

2.5 Why Indonesia First

Indonesia is the world's fourth-most-populous country and the largest economy in Southeast Asia. It has:

- A young, mobile-first population with high smartphone penetration.

- A large unbanked or underbanked population (~40% by some estimates).
- A strong informal economic structure (cooperative banking, community lending, microfinance) that maps naturally to blockchain settlement.
- Remittance flows growing on the scale of tens of billions of dollars per year.
- A regulatory environment that, while still maturing, has shown willingness to engage with crypto and blockchain.
- A meaningful local crypto-aware developer community.

The combination of large unmet demand, technological readiness, and cultural fit makes Indonesia a natural starting point. Once a settlement layer is durable and useful in this market, expansion to comparable markets in Southeast Asia and beyond becomes straightforward.

This is not an "emerging-markets" framing. It is a specific-market framing. Sentrix is built to serve a specific set of users with specific needs, and is designed to scale outward from there.

3 Design Principles

Six principles shape every architectural decision in Sentrix:

1. **Settlement over speculation.** Blockchains optimized for trading create incentive structures that produce trading. Blockchains optimized for settlement produce real economic commerce. We choose the latter at every fork.
2. **Native primitives over contract overhead.** Operations that nearly every user will perform—holding tokens, staking, claiming rewards, moving assets—should not require deploying or interacting with smart contracts. They are part of the protocol.
3. **EVM compatibility for the general case.** Where programmability is needed, the EVM serves it well—a developing standard with broad tooling support. Sentrix runs the EVM faithfully, alongside its native operations.
4. **Deflationary monetary policy.** Inflationary tokens reward early holders at the cost of later holders. Sentrix is supply-capped, halves on the same cadence as Bitcoin, and burns half of every fee. As activity grows, supply contracts.
5. **Crypto-economic security through staking.** Decentralization is a property of who can stake, not of who currently is. Sentrix's validator set is open, permissionless, and economically secured by stake-at-risk.
6. **Real over nominal.** Real-world assets, real businesses, real economic activity. Sentrix avoids on-chain abstractions whose value derives only from other on-chain abstractions. The chain serves the world; the world does not serve the chain.

4 Architecture

Sentrix is structured as four integrated subsystems operating on a single canonical state.

4.1 Consensus Layer

Sentrix uses a delegated-proof-of-stake validator-selection model paired with a three-phase Byzantine Fault Tolerant agreement protocol [3].

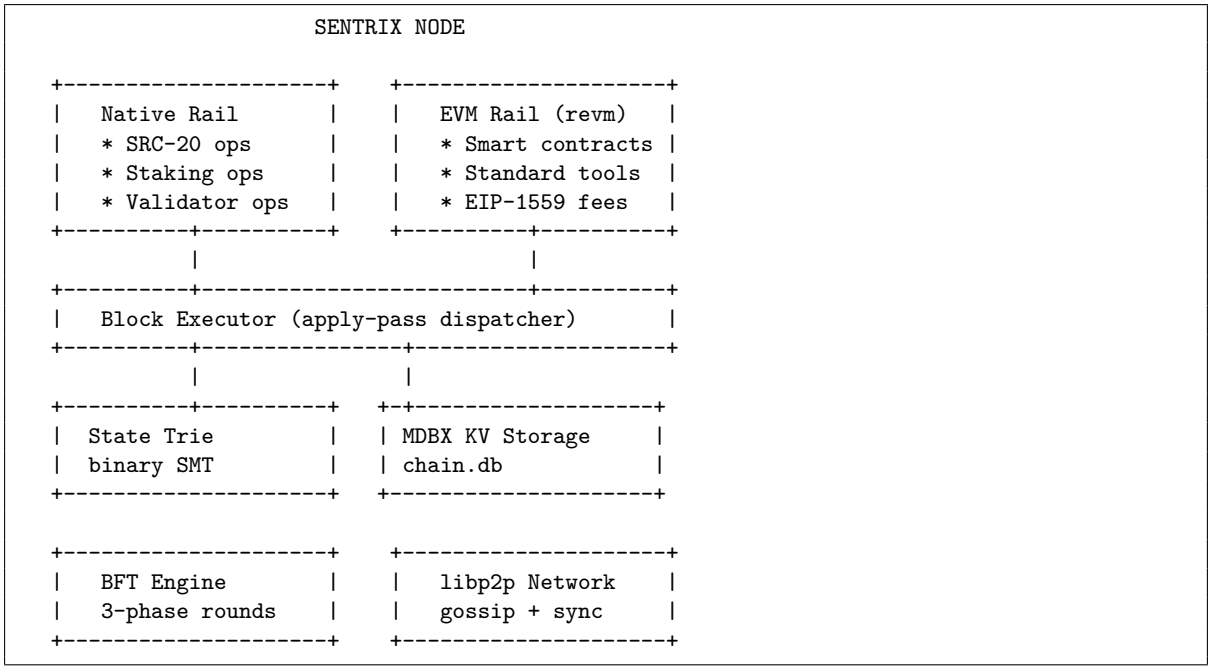

Figure 1: System architecture: two execution rails, one canonical state.

Validator Selection

The validator set is selected by stake-weighted delegation. Any token holder may delegate to any candidate validator; the resulting weighted ranking determines the active set for each epoch. Active-set membership is recomputed at fixed epoch boundaries, allowing new validators to enter and underperforming validators to exit without governance intervention.

A minimum self-stake for validators is enforced at the protocol level. Delegated stake is bonded with an unbonding period: stake withdrawal initiates a fixed-duration timeout during which the stake remains slashable but is not eligible for new rewards. This prevents validators from offloading risk immediately before misbehaving.

Three-Phase Rounds

Within an epoch, the active set runs Tendermint-style three-phase rounds to finalize each block. A round consists of three message types and three corresponding phase transitions:

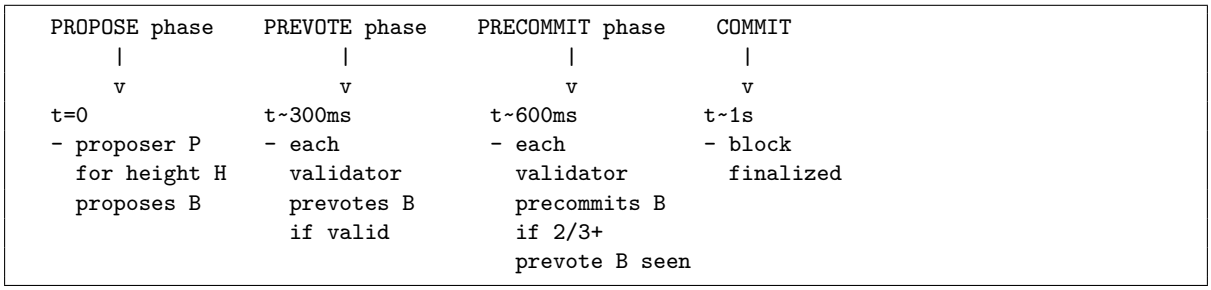

Figure 2: BFT round timing: one second from proposal to finalization.

A block is final when a supermajority ($\geq 2/3$ of stake-weighted active set) precommits the same

block in the same round. The justification—the set of precommit signatures—is included in the next block, providing public proof that the parent was finalized.

Locking and Skip Rounds

After a validator prevotes a block in a round, it locks on that block. In subsequent rounds at the same height, it will only prevote a different block if it observes a "polka" ($\geq 2/3$ prevotes for a new block) at a higher round. This locking property is what guarantees safety: two conflicting blocks cannot both gather supermajority precommits.

If a round fails to finalize within its timeout (proposer offline, network partition, validator absence), the protocol advances to the next round. Proposers rotate round-robin through the active set. After a configurable number of consecutive failed rounds, round timeouts double, providing weak-synchrony recovery.

Safety and Liveness

Under the standard BFT assumption that fewer than one-third of stake-weighted active validators are Byzantine, two safety theorems hold:

- **Agreement.** No two honest validators finalize different blocks at the same height.
- **Validity.** A finalized block is well-formed and applies cleanly.

Liveness holds under a weak-synchrony assumption: messages between honest validators eventually deliver, possibly with bounded delay. Under this assumption, the protocol guarantees that progress eventually occurs.

If the assumption is violated ($\geq 1/3$ Byzantine power), safety is no longer guaranteed and the chain may fork. Recovery in this case requires a social-coordination mechanism—identifying the canonical chain through external trust signals—as in any BFT system.

4.2 Execution Layer

Two execution rails operate on the same canonical state.

Ethereum Virtual Machine. Sentrix runs the EVM via a high-performance Rust implementation [4]. Standard Ethereum contracts (ERC-20, ERC-721, Uniswap-style pools, lending protocols) deploy without modification. Standard tooling—Foundry, Hardhat, MetaMask, ethers, viem—works directly. Gas accounting follows the EIP-1559 model: a per-block base fee that adjusts with demand, plus a sender-set tip paying the proposer for inclusion.

Native protocol operations. Token issuance (`SRC-20`), staking (delegate, undelegate, claim rewards), and validator coordination are not smart contracts but transactions interpreted directly by the protocol. They consume less gas than equivalent contract-based operations because they bypass the EVM entirely. They cannot be exploited via unaudited code because their behavior is specified at the protocol level.

The two rails interoperate. A user holding native SRC-20 tokens can interact with EVM contracts that read those balances via a canonical balance-query precompile. EVM contracts can authorize native operations through a system-call gateway. The chain's canonical state is a single union of EVM state and native ledger.

4.3 State Layer

Sentrix maintains a binary sparse Merkle tree as the canonical state representation [6]. The state root is committed into every block after a specified activation height, providing a cryptographic guarantee that every node applying the same blocks reaches the same state. Differing state roots produce differing block hashes, and BFT ensures the chain converges on a single canonical history.

State is persisted in MDBX [7], an embedded key-value store optimized for high-throughput random reads. Full state is local to every full node; light clients can verify against the trie root with logarithmic Merkle proofs.

State Transition Function

A Sentrix block applies a sequence of transactions to the prior state to produce a new state. Formally, the state transition function $\text{APPLY}(S, B)$ takes state S and block $B = (\text{header}, \text{txs})$ and produces a new state S' or INVALID:

```
APPLY(S, B):
  1. Verify B's header (parent hash, timestamp, proposer signature,
     justification supermajority on parent).
  2. For each transaction TX in B.txs, in order:
       a. Verify TX.signature over canonical_signing_payload(TX).
       b. Verify TX.nonce == S[TX.from].nonce.
       c. Verify TX.fee >= MIN_TX_FEE.
       d. If S[TX.from].balance < TX.fee + TX.amount: skip (insufficient).
       e. Deduct TX.fee from TX.from.
       f. Burn 50% of TX.fee to BURN_ADDRESS; credit the remaining 50%
          directly to the block proposer's balance. (The block subsidy
          follows a separate path: minted into PROTOCOL_TREASURY at the
          end of the block and accrued pro-rata to precommit signers.)
       g. Dispatch TX by to_address:
          - 0x0000...0000  → native token op (decode data as JSON op).
          - 0x0000...0100  → native staking op (decode data as JSON op).
          - 0x0000...0002  → system / claim path (e.g. ClaimRewards).
          - any other      → EVM dispatch via revm; data is the EVM call
                             payload; gas accounted inside revm.
       h. If apply succeeds: update accounts/registries, increment nonce,
          emit events. If apply fails: revert state changes from this TX
          (the fee debit + 50/50 burn-and-credit still stand).
  3. Recompute the state root from the updated trie.
  4. Verify the recomputed state root matches B.header.state_root.
  5. Return S' if all checks pass; otherwise INVALID.
```

This function is deterministic: any node executing the same S and B produces a bit-identical S'. This determinism is what allows independent nodes to verify each other's claims about chain state.

Light Clients

Light clients do not store full state. They track block headers and verify specific facts by requesting Merkle proofs from full nodes. Given a block header containing the canonical state root, a light client can verify any account balance, contract storage slot, or transaction inclusion using a proof logarithmic in the size of state. This makes mobile wallets, dApp browsers, and resource-constrained integrations practical without compromising security.

4.4 Network Model

Sentrix nodes communicate via a libp2p mesh [8]. The protocol uses three message classes:

- **Block gossip.** Newly finalized blocks propagate through gossipsub on topic `sentrix/blocks/1`.
- **Transaction gossip.** User-submitted transactions propagate on topic `sentrix/txs/1` until included in a block.
- **BFT messages.** Proposals, prevotes, and precommits are direct request-response between validators on topic `sentrix/bft/1`. Rebroadcast amplifies messages that may have been dropped.

Peer discovery uses the Kademlia DHT [9] with seed-peer addresses for bootstrap. Each node also publishes a "validator advertisement" record containing its current libp2p multiaddr, allowing other validators to maintain direct connections without external coordination.

A node joining the network bootstraps as follows:

1. Connect to one or more seed peers (configured at startup).
2. Run a Kademlia DHT walk to populate the peer routing table.
3. Request chain head from peers; identify the longest stake-weighted chain.
4. Sync block-by-block from peers until the current head is reached.
5. Subscribe to gossip topics; begin BFT participation if a validator.

Network failure modes under partition are well-defined: minority partitions halt (cannot reach supermajority); majority partitions continue with reduced active set. After partition heals, the minority detects the longer canonical chain and rejoins.

4.5 Transaction Format

Sentrix uses a single canonical transaction wire format for both the native rail and the EVM rail:

```
Transaction {
    txid:         hex32,    // SHA-256 of canonical signing payload
    from_address: address,  // 20-byte hex, "0x"-prefixed
    to_address:   address,  // recipient, or sentinel for protocol op:
                            //   0x0000...0000  → native token op (SRC-20/721/1155)
                            //   0x0000...0100  → native staking op
                            //   0x0000...0002  → PROTOCOL_TREASURY (claim/system)
                            //   anything else → EVM dispatch (revm)
    amount:       uint64,   // sentri  (1 SRX = 10^8 sentri)
    fee:          uint64,   // sentri  (>= MIN_TX_FEE = 10,000)
    nonce:        uint64,   // sender account sequence
    data:         string,   // empty for plain transfers;
                            // canonical JSON {"op":"...",...} for native ops;
                            // standard EVM call payload for EVM dispatch
    timestamp:    uint64,   // unix seconds (anti-replay window)
    chain_id:     uint64,   // 7119 mainnet, 7120 testnet
    signature:    hex,      // secp256k1 ECDSA, 65 bytes (r, s, v)
    public_key:   hex,      // secp256k1 uncompressed, 65 bytes
}
```

Routing is determined by `to_address`: a small set of sentinel addresses dispatches to the native rail (token, staking, treasury), and any other address dispatches to revm with `data` as the EVM call payload. This keeps the wire format uniform — wallets, indexers, and explorers can decode both rails through one schema. EVM users who submit `eth_sendRawTransaction` hit a translation layer that wraps the standard Ethereum transaction into this canonical Sentrix form before mempool entry; gossip and finalization are identical from there on.

4.6 Signing Payload

The signature commits to a canonical JSON serialization of the transaction's eight content fields: `amount`, `chain_id`, `data`, `fee`, `from`, `nonce`, `timestamp`, `to`. Keys are emitted in lexicographic order from a sorted map, so any node serializing the same transaction produces the same byte string. The byte string is hashed with SHA-256, and the resulting 32-byte digest is signed with secp256k1 ECDSA.

`chain_id` is part of the signed payload (replay-protected across networks), but the format is Sentrix-canonical, not EIP-155 RLP. Wallets supporting Sentrix sign this canonical payload directly; the EVM-side `eth_sendRawTransaction` path verifies the EIP-155 signature on the wrapped Ethereum tx independently before translation. Maximum transaction size is configured at the protocol level; oversized transactions are rejected by the mempool.

5 Native Operations

A central design choice in Sentrix is the promotion of common economic primitives from smart contracts to protocol operations.

5.1 The Native vs. Contract Question

The standard pattern on EVM chains is to implement everything—including the most common operations like token transfer and staking—as smart contracts. The abstraction is uniform: all operations consume gas, all are subject to contract-security audit, all are programmable. This is conceptually clean but exacts three costs.

First, **execution overhead.** A simple token transfer in EVM space requires loading contract bytecode, dispatching to a function selector, performing storage reads/writes through the contract namespace, and emitting events. The minimum cost is ~21,000 gas plus contract execution. The actual computation—decrement sender balance, increment recipient balance—is two storage operations.

Second, **security overhead.** Every contract-implemented primitive must be independently audited. Token contracts have produced billions of dollars in losses through bugs in long-deployed code. Native operations are part of the chain's consensus, audited once, deterministic forever.

Third, **upgrade friction.** Contract-implemented primitives are difficult to upgrade without coordinated migration. Native primitives evolve via hard fork, where the entire network upgrades atomically.

5.2 Native Operation Set

Sentrix's native operations include:

- **`SRC-20` token operations.** Issue fungible tokens, transfer them, approve allowances. Im-

plemented as transaction variants applied directly to the native ledger. Token state lives in the canonical state trie alongside account balances.

- **Staking operations.** Delegate stake to a validator, undelegate (with unbonding period), claim accrued rewards, register as a validator candidate. Applied directly to the stake registry.
- **Native transfers.** The simplest case: move SRX between accounts. Costs the protocol minimum fee (`MIN_TX_FEE` = 10,000 sentri = 0.0001 SRX). 50% burned, 50% to proposer.
- **Validator coordination.** Activation, deactivation, and rotation of validators via stake-weighted selection. No governance contract; the protocol enforces rotation each epoch.

These operations remain fully composable: EVM contracts can invoke them via the system gateway, and any wallet supporting Sentrix can execute them via a single signed transaction.

5.3 Lifecycle of a Transaction

To make the architecture concrete, consider a native SRC-20 transfer of 100 tokens of token `0xTOK` from Alice to Bob.

```
Step 1 - Compose transaction
  Alice's wallet builds:
    from_address = 0xALICE...
    to_address   = 0x0000000000000000000000000000000000000000   (token-op sentinel)
    amount       = 0                                            (no SRX moves)
    fee          = MIN_TX_FEE = 10,000 sentri (= 0.0001 SRX)
    nonce        = Alice's current nonce
    data         = {"op":"transfer","contract":"0xTOK",
                    "to":"0xBOB...","amount":100}              (canonical JSON)
    timestamp    = now()
    chain_id     = 7119
    signature    = secp256k1(SHA256(canonical_json), alice_sk)

Step 2 - Submit
  Wallet sends to any RPC endpoint (sentrix_sendTransaction or
    eth_sendRawTransaction for the EVM rail).
  RPC validates format, signature, balance >= fee + amount.
  Tx enters the mempool, gossips to peers on sentrix/txs/1.

Step 3 - Block inclusion
  Validator V (proposer for the next round) drains mempool.
  V constructs block N+1 containing Alice's tx + other valid txs.
  V proposes block N+1 to the active set.

Step 4 - BFT finalization
  Active validators receive the proposal, verify it.
  Each validator prevotes if the proposal is valid.
  Once >= 2/3 prevotes observed, validators precommit.
  Once >= 2/3 precommits observed, block N+1 is finalized.

Step 5 - State apply
  Each node applies block N+1:
    a. Deduct 10,000 sentri (the fee) from Alice's account.
    b. Burn 5,000 sentri to BURN_ADDRESS (50% of fee).
    c. Credit 5,000 sentri directly to validator V's balance
       (the proposer's variable revenue, immediately spendable).
    d. Decrement Alice's SRC-20 balance for token 0xTOK by 100.
    e. Increment Bob's SRC-20 balance for token 0xTOK by 100.
```

```
    f. Increment Alice's nonce.
    g. Emit Transfer event.
    h. Update state trie root; new root committed in block hash.
       (Separately, at the end of the block, the 1 SRX block subsidy
        is minted into PROTOCOL_TREASURY and accrued pro-rata to the
        precommit signers - they claim via ClaimRewards later.)

Step 6 - Confirmation
  Alice's wallet polls RPC; sees tx in a finalized block.
  Bob's wallet (subscribed via WebSocket sentrix_tokenOps channel) is
    notified. Total time from Step 2 to Step 6: ~1-2 seconds.
```

The cycle is identical in principle for plain SRX transfers (no `data` payload), staking operations (`to_address = 0x0000...0100`, JSON op in `data`), and EVM contract calls (`to_address` = the contract, `data` = the EVM call payload). The difference is which apply path is taken in step 5.

6 Tokenomics

Sentrix's native asset is SRX. Its monetary policy is fixed.

6.1 Supply

The maximum supply is 315 million SRX. There is no governance mechanism to change this. After the maximum is reached through block rewards, no further SRX is created.

6.2 Block Reward and Halving

The base block reward is one SRX. The reward halves every approximately 126 million blocks (∼four years at one-second block time), modeled on Bitcoin's halving cadence.

Era	Years	Block reward	Total issued
1	0–4	1.0000 SRX	126.00M SRX
2	4–8	0.5000 SRX	189.00M SRX
3	8–12	0.2500 SRX	220.50M SRX
4	12–16	0.1250 SRX	236.25M SRX
5	16–20	0.0625 SRX	244.13M SRX
		⋯ (asymptotic)	

Table 1: Halving schedule. Combined with the 63M premine, maximum supply approaches 315M SRX over a ∼24-year horizon.

6.3 Fee Mechanism

Every transaction pays a fee. The fee model differs by execution rail.

Native rail. A flat `MIN_TX_FEE` of 10,000 sentri (0.0001 SRX) per transaction, regardless of operation. Native operations have fixed cost at the protocol level.

EVM rail. Standard EIP-1559 mechanics [10]: a per-block `baseFeePerGas` adjusting with congestion, plus a sender-set tip. `gasUsed` × (`baseFee` + `tip`) is charged.

From every fee paid, on either rail:

- **50% is destroyed forever** by being sent to a verifiable burn address from which no transaction can be produced. The total burned is publicly observable on chain.
- **50% is credited directly to the proposer of the block** that included the transaction (immediate spendable balance). This is the variable component of validator revenue on top of the fixed block subsidy.

The block subsidy itself follows a different path: it mints into a protocol-treasury escrow and accrues pro-rata to the block's precommit signers, who claim through an explicit staking operation (§6.5). Fees and subsidy are intentionally split this way — proposing is paid for in real time so validators see immediate revenue, while the subsidy is escrowed so its supply expansion enters circulation only when claimed.

6.4 Premine and Initial Distribution

A premine of 63 million SRX—20% of the supply cap—was allocated at genesis across four roles. All allocation addresses are public and verifiable on chain:

Role	Amount	Address (current holder)	High-level purpose
Founder	21M SRX	`0x5b5b...279d`	Long-term operating treasury
Early Validator	10.5M SRX	`0x328d...fa47`	Validator incentive pool
Ecosystem Fund	21M SRX	`0xeb70...9f14`	Ecosystem grants and growth budget
Reserve	10.5M SRX	`0x2578...ba97`	Strategic reserve

The remaining 80% of supply (252M SRX) issues through block rewards over approximately 24 years. Detailed sub-allocation policies (grant programs, listing budgets, airdrop campaigns, custody arrangements, vesting commitments) live in the operational tokenomics document published at `sentrixchain.com/docs/tokenomics`; that document evolves as the chain matures, while the four amounts and the cap above do not.

Vesting. The premine has no on-chain vesting schedule. The Founder allocation is operationally treated as a long-term holding by the author, not a tradable position, but this is a behavioral commitment, not a protocol enforcement. Read the operational tokenomics document for the current commitment schedule, audit the addresses on chain, and decide your trust accordingly.

6.5 Block Subsidy Routing

Block subsidies do not pay validators directly. They are credited to a protocol-treasury escrow, then accrued pro-rata into the pending-rewards balance of every validator that signed the block's precommit (delegators inherit their validator's accrual, minus commission). Validators and delegators drain into spendable balance through an explicit staking operation. This design preserves the supply invariant — new SRX enters circulation only when claimed, never when produced — and provides a clean accounting boundary between issuance and distribution.

7 Validator Economics

7.1 Stake at Risk and Slashing

Every validator must bond a minimum self-stake to the protocol. Self-stake is at risk: provable misbehavior produces slashing.

Trigger	Evidence	Stake slashed	Jail duration
Double-sign	Two signatures on conflicting blocks at the same height	20% (parametric)	Permanent (tombstone)
Downtime	Missed > threshold blocks in window	0.1% (parametric)	Configurable

Slashed SRX is destroyed (added to the burn address), not redistributed. This preserves the supply invariant and avoids creating perverse incentives among non-slashed validators to push for slashing competitors.

7.2 Block Reward and Fee Share

A validator's revenue has two components, which flow on different paths:

- **Block subsidy share.** For every block whose precommit they sign, the validator accrues a slice of that block's 1-SRX subsidy proportional to their stake. With N active validators all signing every block, a validator with stake fraction $f = \frac{\text{validator\_stake}}{\text{total\_active\_stake}}$ accrues roughly f of every block's subsidy. The accrual sits in escrow (PROTOCOL_TREASURY) and is drained by an explicit `ClaimRewards` staking operation.
- **Fee share.** For each block the validator *proposes*, 50% of that block's transaction fees credits directly to their balance (immediate spendable). Round-robin proposer rotation, weighted by stake, gives them an expected fraction f of all proposed blocks per epoch.

Expected revenue per epoch, for a validator with stake fraction f:

$$\text{revenue} \approx \underbrace{f \cdot \text{epoch\_length} \cdot \text{block\_subsidy}}_{\text{from signing blocks (escrowed)}} + \underbrace{0.5 \cdot f \cdot \text{epoch\_fees}}_{\text{from proposing blocks (spendable)}}$$

Delegators inherit their validator's accrual on both components, minus a commission rate the validator publishes.

7.3 Liveness Penalty

Validators that miss blocks accumulate downtime against a moving window (`LIVENESS_WINDOW`). Sustained downtime produces jailing, removing the validator from the active set and slashing a small fraction of stake.

7.4 Genesis and Bootstrap

Sentrix's genesis state is constructed from a configuration file (`genesis.toml`) declaring chain ID (7119 mainnet, 7120 testnet), initial premine balances, the initial validator set with public keys and self-stake commitments, and protocol parameters effective from block 1. Any node can verify the genesis state by re-applying the genesis configuration; the resulting state root must match the hardcoded genesis hash.

7.5 Becoming a Validator

The validator set is open and permissionless. Any operator who can run reliable infrastructure and bond minimum self-stake may register.

Concrete requirements:

- **Hardware:** a modern x86-64 server with $\geq$ 4 CPU cores, $\geq$ 8 GB RAM, $\geq$ 250 GB SSD storage, and stable network connectivity.
- **Network:** a stable public IPv4 address with TCP port 30303 reachable for the libp2p P2P transport.
- **Self-stake:** a minimum bond of native SRX, locked while active and during the unbonding period.
- **Identity:** a registered validator wallet (secp256k1 keypair) with a human-readable validator name. The wallet signs blocks and votes; secure key management is the operator's responsibility.

There are no jurisdictional restrictions, no whitelist, no application process. Misbehavior is enforced economically (slashing) rather than administratively.

7.6 Incident Response

Sentrix's incident-response model is structured around three principles:

Detection through observation. Every full node independently verifies block applications. State divergence produces divergent block hashes; nodes with divergent state cannot win the canonical chain.

Recovery through canonical-state alignment. When validators diverge, the recovery protocol identifies the canonical state (the chain hash that supermajority of stake-weighted validators agrees on) and replicates it to the diverged node.

Coordinated upgrade through hard fork. Bug fixes that require consensus changes ship as hard forks gated by activation height. Operators upgrade their binary before the activation height; on activation, all nodes apply the new logic atomically.

8 Security Model

Sentrix's security derives from four layers of guarantee.

8.1 Cryptographic Guarantee

Every transaction is signed with a private key whose public address authorizes the operation. Block headers contain the state root and the proposer's signature. The state root commits to the canonical state via the binary sparse Merkle tree.

8.2 Crypto-Economic Guarantee

Validators participate because it is profitable to do so honestly. Slashing makes dishonest validation negative-expected-value at any stake size.

8.3 Social Guarantee

The chain's behavior is observable. State, blocks, transactions, validator set, slashing events—all are public. Trust is replaced by verification.

8.4 Quantitative Security

The cost of compromising chain safety is bounded below by the cost of acquiring and slashing one-third of the total bonded SRX. If the protocol bonds X SRX at market price P, the minimum attack cost A satisfies:

$$A \geq \frac{X \cdot P}{3} \tag{1}$$

This is a lower bound, not an upper bound. The bound rises monotonically with both X (bonded supply) and P (market price), which means chain success increases attack cost.

8.5 Long-Range Attacks and Weak Subjectivity

Pure proof-of-stake chains are theoretically vulnerable to long-range attacks: an attacker who acquires old validator keys can fork the chain from an arbitrarily early point. Sentrix defends against this through three mechanisms: an unbonding period during which stake remains slashable, weak-subjectivity checkpoints from which new nodes start their sync, and periodic active-set re-sync at intervals shorter than the unbonding period [11].

8.6 Failure Modes

The chain has four well-defined failure modes: more than 1/3 Byzantine stake (safety not guaranteed); more than 2/3 Byzantine stake (invalid state can be signed); network partition (minority halts, majority continues); coordinated censorship (resistant via proposer rotation).

8.7 Privacy Posture

Sentrix is transparent by design. Every transaction, balance, and contract call is publicly observable. This is a deliberate choice: the chain serves real-economy settlement, where audit trails are a feature rather than a liability. Users requiring privacy for specific use cases can build it on top: zk-rollup contracts, privacy-preserving dApps, off-chain commitments verified on-chain. The base layer remains observable; privacy is opt-in at the application layer.

9 The Real-Economy Thesis

Sentrix's positioning is real-world settlement.

Real-world assets. Tangible and contractual rights—real estate, invoices, equity in private companies, rights to revenue streams, bills of lading—are valuable to their holders. They are also illiquid. A blockchain that can represent these as on-chain tokens, with provable provenance and atomic settlement, removes friction proportional to the value of the asset.

Local-currency settlement. Most economic activity is denominated in local currencies—rupiah, peso, dong, baht, ringgit, dirham. A blockchain that supports stablecoin issuance against these currencies, with native operations for cheap fast transfer, becomes a settlement rail for retail commerce that does not currently have one.

Cross-border commerce and remittance. Cross-border payments traditionally settle through correspondent banking with multi-day delays and disproportionate fees. A blockchain whose native operations cost fractions of a cent and finalize in one second is fundamentally suited to small-size cross-border flow. Indonesia is a remittance-receiving country at the scale of tens of billions of dollars per year.

Microfinance and cooperative settlement. Group savings (*arisan*), cooperative banking

(*koperasi simpan-pinjam*), and community lending are deeply embedded in Indonesian economic culture. They operate on trust, paper records, and informal coordination. A blockchain that can represent these flows—on-chain, verifiable, low-cost—is an upgrade rather than a replacement.

10 Comparison to Prior Work

	Block time	Finality	Consensus	VM	Supply policy
Bitcoin [1]	∼10 min	Probabilistic	PoW	None	Capped, halving
Ethereum [2]	∼12 sec	Probabilistic →	PoS	EVM	Inflationary
Solana [5]	∼400 ms	Probabilistic	PoH+TowerBFT	SVM	Inflationary
Cosmos [3]	∼6 sec	Single-block	Tendermint	SDK	Inflationary
Polygon	∼2 sec	Probabilistic	PoS BFT	EVM	Inflationary
Aptos	∼250 ms	Single-block	AptosBFT	Move	Inflationary
Sui	∼390 ms	Single-block	Mysticeti	Move	Inflationary
Near	∼1.2 sec	Single-block	Nightshade	Wasm	Inflationary
Sentrix	**∼1 sec**	**Single-block**	**DPoS+BFT**	**EVM+Native**	**Capped+halving+burn**

Sentrix is not novel in any single dimension. Its contribution is the combination: native-EVM hybrid execution, stake-weighted BFT, deflationary capped supply, sub-second finality, oriented toward real-world settlement and rooted in the Indonesian market.

11 Governance

11.1 Current State

Sentrix's protocol upgrades are coordinated through binary releases gated by activation height. The author releases new node binaries; operators upgrade in the time window before a fork's activation height; on activation, all nodes apply the new logic atomically. This is the same mechanism Bitcoin, Ethereum, and most Tendermint-based chains use for hard forks.

11.2 The SentrixSafe Multisig

Authority over privileged operations (validator authority key, treasury reserve management, fee-fork toggle) is held by the SentrixSafe multisig, a Gnosis-Safe-derived contract deployed onto the chain shortly after Voyager activation as part of the canonical contracts set. Currently configured 1-of-1 with the author as sole signer; intended to expand organically to N-of-M as the chain attracts long-term contributors who can credibly sign off on protocol-authority operations. There is no hard timeline; the bar is "credible co-signer with operational continuity and skin-in-the-game," not "calendar quarter."

11.3 Future On-Chain Governance

Stage 5 of the path forward migrates protocol decisions onto a stake-weighted on-chain governance mechanism: anyone holding above a minimum stake may submit a proposal; voting is stake-weighted across the active validator set; proposals require minimum participation (target 33%); pass thresholds vary by proposal type; passing proposals trigger pre-defined on-chain effects.

11.4 What Cannot Be Governed

Some chain properties are deliberately non-governable: the 315M SRX supply cap, the four-year halving schedule, the 50% fee burn ratio, and the genesis allocation. No vote, no fork, no upgrade changes these. They are the parameters where stability is the feature.

12 Path Forward

We describe the trajectory of Sentrix in stages rather than dates.

- **Stage 1: Mainnet operating.** Small geographically distributed validator set; native and EVM execution working; block explorer, faucet, dev tooling; source-available codebase. *(Current.)*
- **Stage 2: Liquidity and discovery.** First on-chain market for SRX; first contracts deployed by external developers; chain-registry indexing; recognition by standard wallets and bridges.
- **Stage 3: Real-economy integrations.** First production use cases involving real-world assets, local-currency settlement, or cross-border flow.
- **Stage 4: Validator decentralization.** Growth from a small bootstrap to a meaningful number of independent operators across multiple jurisdictions.
- **Stage 5: On-chain governance.** Migration of meaningful protocol decisions onto stake-weighted governance.
- **Stage 6: Open-license activation.** BUSL-1.1 transitions to Apache 2.0 after the Change Date.

These stages do not run on a calendar. They run on the satisfaction of preconditions.

13 Conclusion

Sentrix is a deliberate response to a structural absence. The dominant blockchains of the present moment serve trading and speculation well, and they serve real economic settlement poorly. We do not believe this is inevitable. We believe it is the consequence of design choices that can be made differently.

Sentrix's design choices are: native operations for common primitives, EVM compatibility for the general case, stake-weighted Byzantine Fault Tolerant consensus, sub-second finality, capped halving deflationary supply, and a market-entry orientation toward the Indonesian economy and the broader unmet demand for real-world settlement infrastructure.

Sentrix is open to use, open to extension, and open to inspection. Real value, real assets, real economic activity—on chain, fast, final, and durable.

A Protocol Parameters

B Risk Disclosures

This appendix documents known risks. It is not exhaustive; readers must perform their own due diligence.

Parameter	Value	Notes
BLOCK_TIME	∼1 s	Target; actual varies with round duration
MAX_TX_PER_BLOCK	5,000	Configurable at protocol level
MAX_SUPPLY	315,000,000 SRX	Hard cap, no governance override
INITIAL_BLOCK_REWARD	1 SRX	Halves every HALVING_PERIOD
HALVING_PERIOD_BLOCKS	∼126,000,000	∼4 years at 1 s blocks
MIN_TX_FEE	10,000 sentri	0.0001 SRX (native rail)
BURN_RATIO	50%	Of every transaction fee
MIN_VALIDATOR_SELF_STAKE	parametric	Configured at genesis
UNBONDING_PERIOD	parametric	Window stake-slashable after withdrawal
LIVENESS_WINDOW	14,400 blocks	∼4 hours at 1 s blocks
JAIL_DURATION_BLOCKS	600	∼10 minutes at 1 s blocks
DOWNTIME_SLASH_BP	10 (0.1%)	Of self-stake at jail
EPOCH_LENGTH	parametric	Active-set rotation cadence

Technical risks. Single Rust implementation: a bug can affect the entire network until patched; multi-implementation diversity is not present. Active validator set is small at this stage; sustained Byzantine behavior by a supermajority is theoretically possible until the active set grows. The on-chain consensus-jail dispatch is currently disabled (`JAIL_CONSENSUS_HEIGHT` set to a sentinel that prevents activation) following two production incidents traced to non-determinism in liveness tracking; jailing for downtime and double-sign is still available through the manual operator path until the dispatch is re-engineered and re-enabled by hard fork.

Economic risks. SRX is not currently trading on any exchange. There is no market price. The 315M cap is a protocol constant, not a market valuation. Until SentrixSafe expands to N-of-M and/or Founder allocation is on-chain vested, the Founder address can move 21M SRX at any time.

Regulatory risks. The Indonesian regulatory landscape continues to evolve. Holders and validators in different jurisdictions face different regulatory regimes. SRX is intended as a utility token; whether any specific jurisdiction classifies it as a security depends on local law.

Operational risks. Sentrix is solo-built. The author's continued participation is not guaranteed by any contract. Long-term resilience depends on the codebase being open enough that other operators can run forks (BUSL → Apache 2.0 after Change Date).

C Legal Notice

This whitepaper is a description of a software protocol. It is not an offering document, prospectus, investment solicitation, or financial advice. SRX is a utility token used to pay transaction fees, secure the chain through staking, and (in the future) participate in governance.

The Sentrix Chain protocol is open-source infrastructure under the Business Source License 1.1, transitioning to Apache 2.0 after the Change Date specified in the LICENSE file.

The author makes no representations about future SRX market price, ecosystem adoption, partnership outcomes, or regulatory acceptance. Forward-looking statements describe intent, not guarantees. Readers are responsible for compliance with their local laws and regulations.

About the Author

Satya Kwok built Sentrix Chain solo in Rust. The author's prior work and engagement is publicly observable on GitHub (`@satyakwok`) and through the project's open commit history at `github.com/sentrix-labs/sentrix`. The author is contactable at `satya@sentrixchain.com`.

References

[1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.*

[2] Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.*

[3] Buchman, E., Kwon, J. (2016). *Tendermint: Byzantine Fault Tolerance in the Age of Blockchains.*

[4] Bluealloy (2022). *revm: Rust Ethereum Virtual Machine.*

[5] Yakovenko, A. (2017). *Solana: A new architecture for a high performance blockchain.*

[6] Merkle, R. (1980). *Protocols for Public Key Cryptosystems.*

[7] *libmdbx: Memory-mapped key-value store.* `github.com/erthink/libmdbx`

[8] *libp2p: Modular peer-to-peer networking stack.* `libp2p.io`

[9] Maymounkov, P., Mazières, D. (2002). *Kademlia: A Peer-to-peer Information System Based on the XOR Metric.*

[10] Buterin, V. et al. (2019). *EIP-1559: Fee market change for ETH 1.0 chain.*

[11] Buterin, V., Griffith, V. (2017). *Casper the Friendly Finality Gadget.*